SMATCH - Smart Async Task Chaining

Documentation Framework Asynchrone Salesforce

Version : février 2026 – Dépôt : 20260215

Table des matières

1. Introduction
2. Choix d'implémentation
3. Manuel Administrateur
4. Manuel du Développeur
5. Annexe

I. Introduction

I.A. Vision

SMATCH (SMart Async Task CHaining) est un framework asynchrone pour Salesforce qui permet d'orchestrer, d'exécuter et de surveiller des tâches en arrière-plan de façon fiable et efficace. L'objectif est d'offrir une base réutilisable pour tous les besoins d'exécution différée (emails, intégrations, traitements volumineux, maintenance) sans multiplier les mécanismes ad hoc propres à chaque projet.

I.B. Problème adressé

Les organisations Salesforce ont souvent besoin de :

- Exécuter des traitements lourds sans bloquer les utilisateurs
- Planifier des tâches à des moments précis ou récurrents
- Gérer des volumes importants tout en respectant les limites de la plateforme (CPU, requêtes, DML)
- Suivre l'état d'exécution et diagnostiquer les erreurs
- Adapter le comportement sans redéployer du code

Sans un framework centralisé, chaque équipe développe des solutions isolées (Queueable, Schedulable, Batch) qui se répètent, sont difficiles à faire évoluer et manquent souvent de visibilité.

I.C. Objectifs du framework

I.C.1 Simplicité pour le développeur

- **Interface unique** : implémenter IAsyncTask avec une méthode run(SF_Async__c) suffit à créer une tâche exécutable
- **Enregistrement des tâches** via des enregistrements SF_Async__c (horodatage, ordre, classe, cible)
- **Aucune gestion manuelle** de la planification : le Worker prend en charge la mise en file et l'exécution

I.C.2 Performance et robustesse

- **Smart batching** : taille des lots calculée dynamiquement à partir des métriques historiques (CPU moyen par tâche)
- **Limites par tâche et par classe** : Tasks_Per_Worker_Limit__c, Max_Tasks_Per_Run_Learned__c (erreurs callout/DML)
- **Respect des limites** : arrêt anticipé quand le CPU ou le DML approche de la limite, marge de sécurité configurable
- **Regroupement par classe**, FOR UPDATE pour éviter le traitement en double

I.C.3 Observabilité

- Logs détaillés par exécution (CPU, SOQL, Heap) dans SF_Async_Log__c
- Statistiques de performance par classe (SF_Async_Performance__c)
- Événements et alertes en cas d'erreur (Platform Event Async_WakeUp__e)
- Diagnostic invocable affichant l'état complet du framework

I.C.4 Configuration sans déploiement

- Paramètres centralisés dans une Custom Metadata (Async_Framework_Configuration__mdt)
- Overrides runtime via Custom Settings (Async_Runtime_Config__c)
- Ajustement des seuils, des fréquences et des tailles de lot sans déploiement Apex
- Interface Flow pour piloter le framework

I.C.5 Continuité de service

- **Watchdog** (AsyncScheduler) pour détecter l'absence de Worker actif et déclencher un réveil
- **Reprogrammation automatique** du Worker selon la fréquence configurée
- **Nettoyage automatique** des enregistrements anciens pour limiter l'empreinte en base
- **Force Start** pour relancer manuellement le système

I.D. Public cible

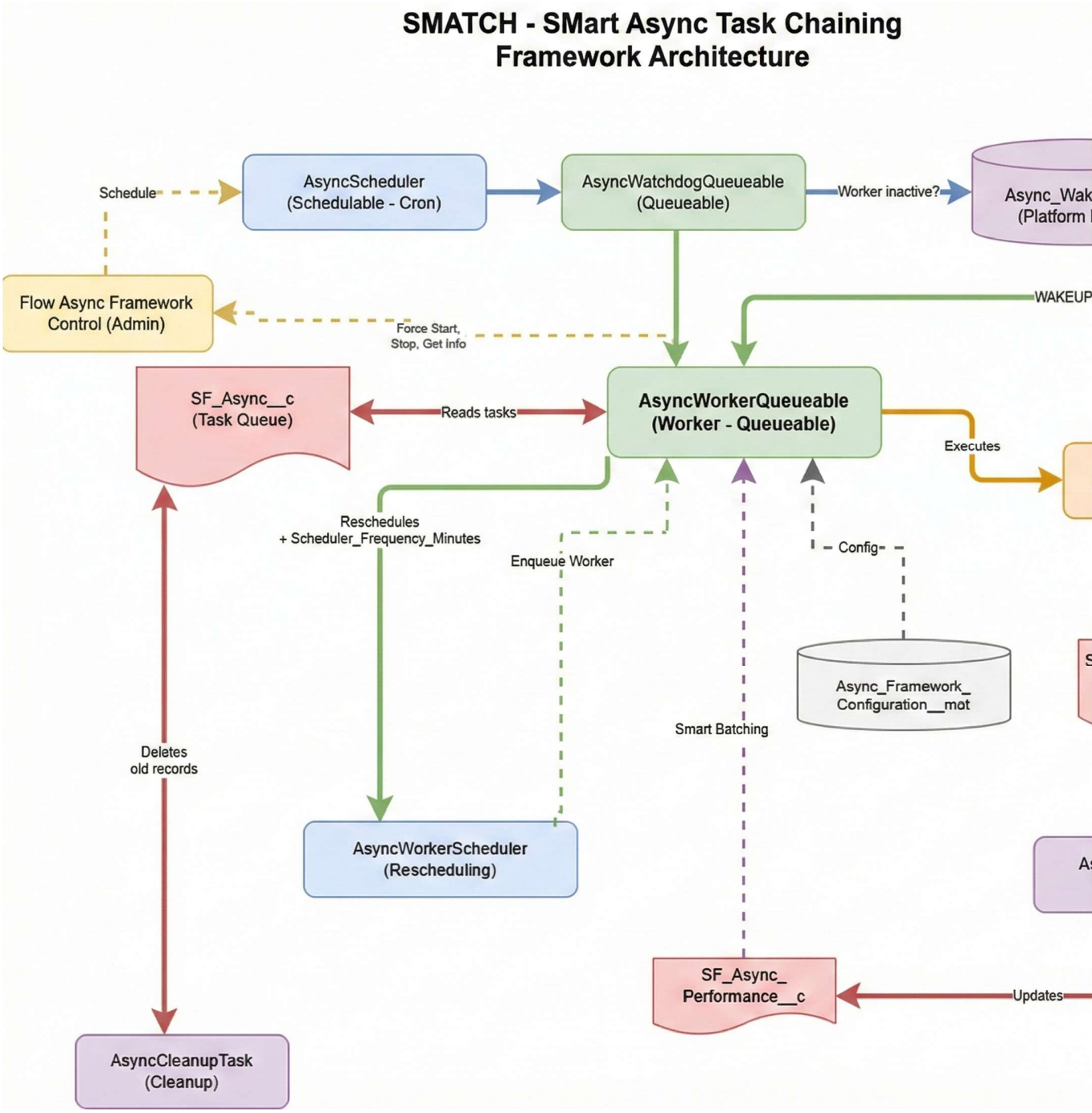
Rôle	Usage
Développeurs Salesforce	Intégration rapide de tâches asynchrones via l'interface IAsyncTask
Architectes	Standardisation des traitements asynchrones à l'échelle de l'org
Administrateurs	Surveillance et paramétrage via le Flow sans développement
Équipes support	Diagnostic et relance via l'interface Smatch

I.E. Bénéfices attendus

Bénéfice	Description
Réutilisation	Un seul framework pour toutes les tâches asynchrones du projet
Maintenabilité	Configuration et évolution centralisées
Visibilité	Suivi du backlog, des performances et des erreurs
Fiabilité	Limites respectées, traitement des erreurs, reprise automatique
Évolutivité	Batch d'apprentissage pour ajuster dynamiquement les performances

I.F. En résumé et Architecture

SMATCH vise à devenir le **socle standard** pour l'exécution asynchrone sur Salesforce : une solution unique, configurable et observable, qui réduit la dette technique et améliore la qualité des traitements en arrière-plan.



Flux principal :

6. **AsyncScheduler** (cron) → exécute **AsyncWatchdogQueueable**
7. **AsyncWatchdogQueueable** → vérifie s'il y a des Workers actifs ; sinon publie Async_WakeUp__e (WAKEUP)
8. **AsyncWakeUpTrigger** (after insert sur Async_WakeUp__e) → enqueue **AsyncWorkerQueueable**

9. **AsyncWorkerQueueable** → lit SF_Async__c, exécute les tâches IAsyncTask, logue, reprogramme **AsyncWorkerScheduler**
10. **AsyncLearningBatch** → alimente SF_Async_Performance__c pour le Smart Batching

Vue d'ensemble des classes : voir `docs/drawio/AsyncFramework_Overview.drawio`

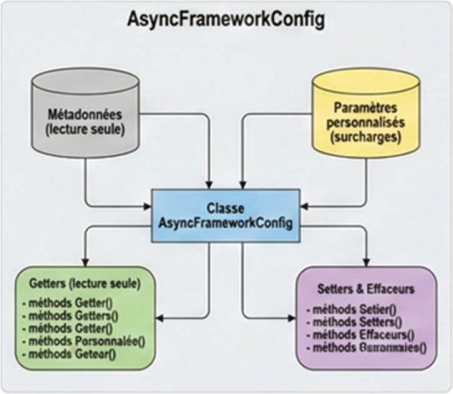
II. Choix d'implémentation

Ce chapitre détaille les décisions techniques et la description des méthodes, triggers et configurations. Chaque section commence par une justification du choix puis décrit les éléments en détail.

II.1. Configuration Custom Metadata

Justification : Les constantes (limite CPU, taille des lots, fréquence du scheduler) doivent être modifiables sans déploiement. La Custom Metadata avec getInstance() ne consomme pas de requête SOQL.

AsyncFrameworkConfig



Classe utilitaire qui lit la configuration depuis Async_Framework_Configuration__mdt.Default_Configuration. Fournit des valeurs par défaut si les métadonnées ne sont pas déployées.

Méthode	Entrées	Sorties	Description
`clearConfig()`	Aucune	void	Vide le cache après mise à jour des métadonnées
`getSafeCpuLimit()`	-	Integer	Limite CPU sûre en ms (défaut 45000)

`getMinBatchSize()`	-	Integer	Taille minimale de lot (défaut 1)
`getMaxBatchSize()`	-	Integer	Taille max (lit d'abord `Async_Runtime_Config__c.Max_Batch_Size_Override__c`)
`getMaxNumberWorker()`	-	Integer	Nombre max de workers (défaut 32)
`getParallelLowThreshold()`	-	Integer	Seuil pour déclencher la parallélisation (défaut 200)
`getTasksPerWorkerDefault()`	-	Integer	Défaut tâches max par Worker (200)
`getDefaultCpuCostMs()`	-	Integer	Coût CPU par défaut par tâche (1000)
`getCpuReserveMs()`	-	Integer	Réserve CPU avant les limites (2000)
`getDmlSafeThreshold()`	-	Integer	Seuil DML avant réduction de lot (75)
`getDaysToAnalyze()`	-	Integer	Jours pour les statistiques (7)
`getDaysToKeep()`	-	Integer	Jours avant suppression des anciennes tâches (30)
`getDefaultSchedulerFrequencyMinutes()`	-	Integer	Fréquence du scheduler (1)
`setMaxBatchSizeOverride(Integer)`	newSize	void	Override runtime (Custom Settings)
`clearMaxBatchSizeOverride()`	-	void	Restaure la valeur metadata

Async_Framework_Configuration__mdt (champs principaux)

Champ	Type	Défault	Description
Before_Cancelling__c	Number	3	Nombre de tentatives avant l'annulation d'une tâche
CPU_Reserve_ms__c	Number	2000	Marge de sécurité du processeur (ms) réservée dans la b
Cleanup_Batch_Size__c	Number	2000	Taille du lot pour les opérations de suppression de netto
Cleanup_CPU_Reserve_ms__c	Number	5000	Marge de sécurité du processeur (ms) pour la tâche de r
DML_Safe_Threshold__c	Number	75	Seuil de lignes DML avant que le worker ne cesse de pub (max 150)
Days_To_Analyze__c	Number	7	Nombre de jours à analyser pour les performances histo
Days_To_Keep__c	Number	30	Nombre de jours pendant lesquels conserver les docum
Delete_Pending_Max_Iterations__c	Number	10	Nombre maximal d'itérations pour l'opération de suppre
Default_CPU_Cost_ms__c	Number	1000	Coût CPU par défaut (ms) lorsqu'aucune donnée de perf
Execution_Order__c	Number	99	Ordre d'exécution des tâches de nettoyage
Max_Batch_Size__c	Number	200	Nombre maximal de tâches par lot
Max_Number_Worker__c	Number	32	Nombre maximal de travailleurs parallèles autorisés
Min_Batch_Size__c	Number	1	Nombre minimum de tâches par lot
Parallel_Low_Threshold__c	Number	200	Lorsque effectiveLimit ≤ valeur, des processus parallèle
Safe_CPU_Limit__c	Number	45000	Marge de sécurité du processeur : 45 s sur 60 s disponib
Safety_Margin_Multiplier__c	Number	1	Multiplicateur appliqué au temps CPU moyen
Scheduler_Frequency_Minutes__c	Number	1	Fréquence d'exécution du planificateur en minutes

Status_Success__c	Text	Success	Valeur d'état pour une exécution de tâche réussie
Tasks_Per_Worker_Default__c	Number	200	Nombre maximal de tâches par nœud de calcul par défau SF_Async__c
Worker_number__c	Picklist	1	Nombre de travailleurs à démarrer

Champ	Type	Défaut	Description
Safe_Cpu_Limit__c	Number	45000	Limite CPU en ms
Min_Batch_Size__c	Number	1	Lot minimum
Max_Batch_Size__c	Number	200	Lot maximum
Max_Number_Worker__c	Number	32	Workers max en parallèle
Parallel_Low_Threshold__c	Number	200	Seuil pour parallélisation
Scheduler_Frequency_Minutes__c	Number	1	Fréquence Worker
DML_Safe_Threshold__c	Number	75	Seuil DML (sur 10000)
Days_To_Keep__c	Number	30	Conservation des données
Cleanup_Batch_Size__c	Number	2000	Lot du nettoyage

II.2. Platform Event et Trigger

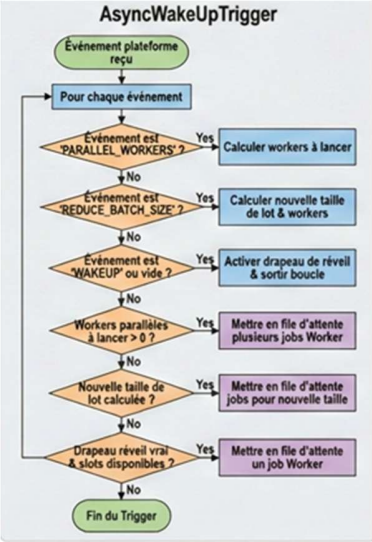
Justification : L'événement Async_WakeUp__e permet de réveiller le Worker en temps réel lorsqu'une tâche est insérée ou qu'un système externe le demande. Le trigger enqueue le Worker sans consommation de polling.

Async_WakeUp__e (Platform Event)

Champ	Type	Description
Event_Type__c	Text	WAKEUP, REDUCE_BATCH_SIZE, PARALLEL_WORKERS, ERROR
Batch_Size__c	Number	Taille de lot (REDUCE_BATCH_SIZE, PARALLEL_WORKERS)
Workers_Count__c	Number	Nombre de workers (REDUCE_BATCH_SIZE)
Error_Message__c	Text	Message ou entier parsable (REDUCE_BATCH_SIZE)
Class_Name__c	Text	Classe concernée
Related_Record_Id__c	Text	ID SF_Async__c
Target_Record_Id__c	Text	Cible

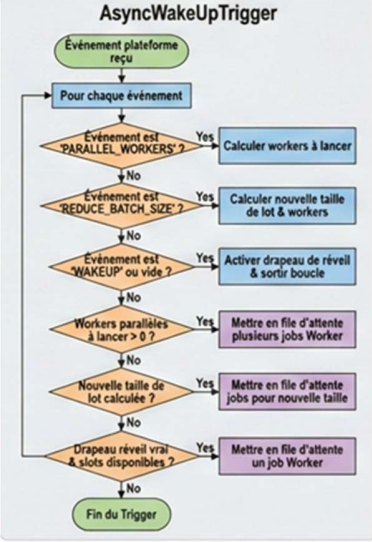
AsyncWakeUpTrigger

Contexte : after insert sur Async_WakeUp__e.



Logique :

- **PARALLEL_WORKERS** : enqueue jusqu'à min(Batch_Size__c, 64, 50) Workers
- **REDUCE_BATCH_SIZE** : met à jour Async_Runtime_Config__c, enqueue des Workers supplémentaires
- **WAKEUP** ou vide : enqueue 1 Worker si des slots Queueable sont disponibles

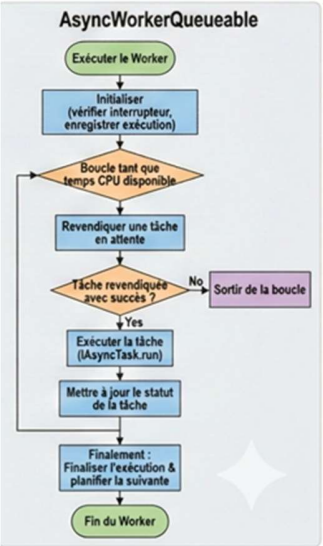


II.3. Worker et Smart Batching

Justification : Le Worker doit adapter dynamiquement la taille des lots selon les performances historiques et les limites de la plateforme. Le Smart Batching évite les erreurs et optimise le débit.

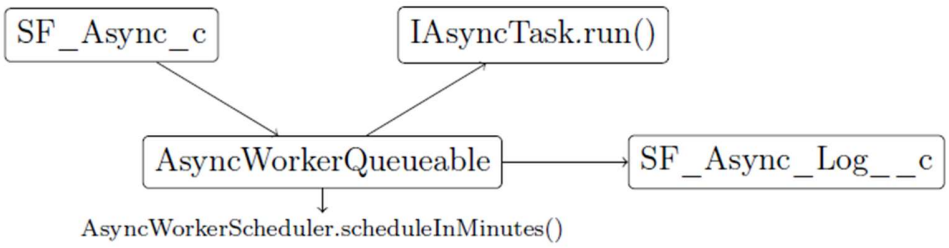
AsyncWorkerQueueable

Entrée : QueueableContext (Salesforce)



Comportement :

11. Sélectionne la prochaine classe de tâches (SF_Async__c non exécutées, tri par Scheduled_Time__c, Execution_Order__c)
12. Calcule effectiveLimit = min(limite CPU/coût moyen, Tasks_Per_Worker_Limit, Max_Tasks_Per_Run_Learned)
13. Sélectionne les tâches avec FOR UPDATE
14. Exécute chaque tâche via IAsyncTask.run()
15. En cas d'erreur callout/DML : apprend Max_Tasks_Per_Run_Learned__c = 1
16. En cas d'approche limite DML : publie REDUCE_BATCH_SIZE
17. Reprogramme via AsyncWorkerScheduler ou spawn des Workers parallèles

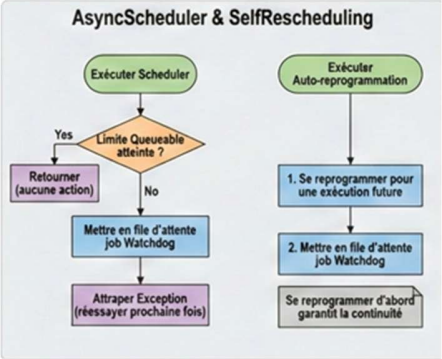


II.4. Scheduler et Watchdog

Justification : Deux modes de planification (multi-cron ou single self-rescheduling) selon les besoins. Le Watchdog garantit la continuité même en cas de défaillance.

AsyncScheduler

Schedulable exécuté par des jobs cron. Enqueue **AsyncWatchdogQueueable**.



Méthode : execute(SchedulableContext) → enqueue Watchdog

AsyncSchedulerSelfRescheduling

Un seul job qui se replanifie après chaque exécution. Utilisé en mode « single ».

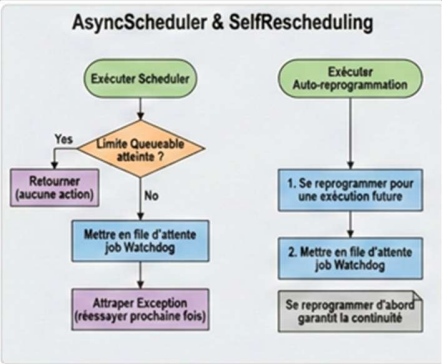
AsyncWatchdogQueueable

Vérifie si des Workers sont actifs (query AsyncApexJob). Si non, publie Async_WakeUp__e(Event_Type__c = 'WAKEUP').

AsyncWorkerScheduler

Reprogramme le Worker avec une fréquence en minutes (depuis metadata ou paramètre).

Méthode	Entrées	Sorties	Description
`scheduleInMinutes(Integer)`	freq ou null	String (jobId)	Schedule le Worker, null = metadata

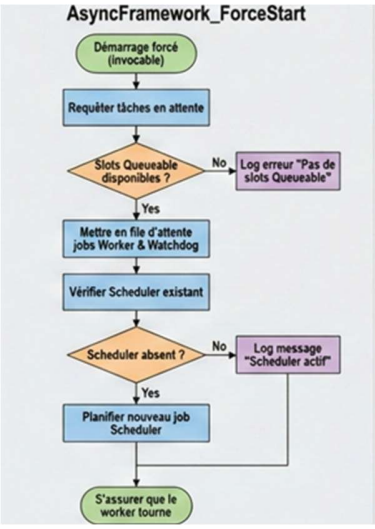


II.5. Actions Flow invocables

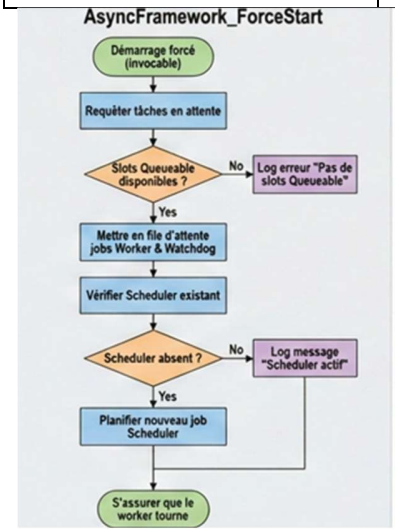
Justification : Permettre aux administrateurs de piloter le framework sans code ni déploiement.

AsyncFramework_ForceStart

Démarre le Worker et le Watchdog. Planifie le Scheduler si absent.



Sortie	Type	Description
message	String	État des tâches, Worker, Watchdog, Scheduler



AsyncFramework_Diagnostic

Affiche l'état du framework : tâches en attente, logs d'erreur, Workers, Schedulers, tâches Statistics/Cleanup planifiées.

Sortie	Type	Description
message	String	Rapport textuel complet

AsyncFramework_StopSchedulers

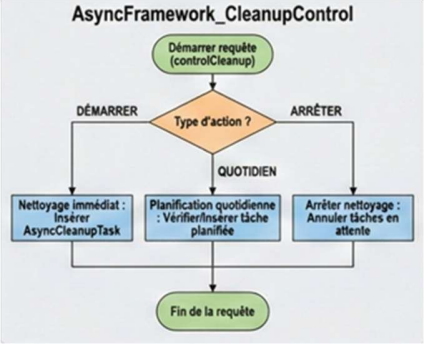
Arrête tous les Schedulers Async (Async Framework Scheduler, AsyncWorker).

Sortie	Type	Description
message	String	Liste des jobs arrêtés ou erreurs

AsyncFramework_CleanupControl

Contrôle le nettoyage des données. Actions : START (immédiat), START_DAILY (planification quotidienne), STOP (annulation).

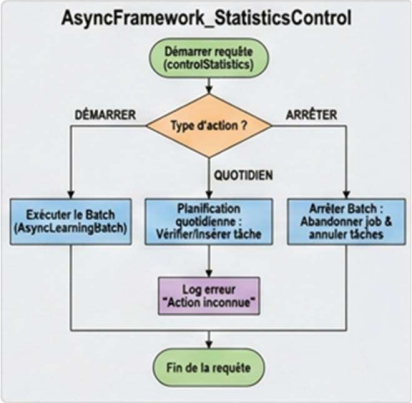
Entrée	Type	Description
action	String	START, START_DAILY, STOP
Sortie	**Type**	**Description**
message	String	Résultat de l'action



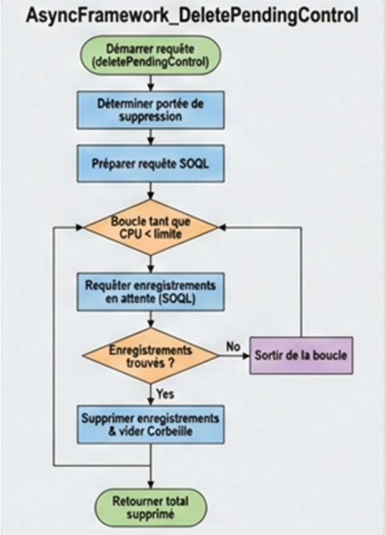
AsyncFramework_StatisticsControl

Contrôle les statistiques de performance. Actions : START, START_DAILY, STOP.

Entrée	Type	Description
action	String	START, START_DAILY, STOP
Sortie	**Type**	**Description**
message	String	Résultat

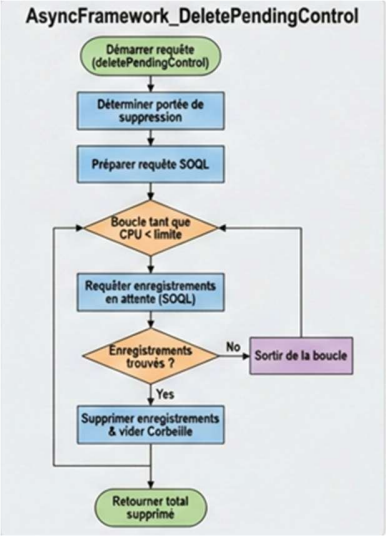


AsyncFramework_DeletePendingControl / AsyncFramework_DeletePendingFlow



Supprime les tâches en attente. Option : tout supprimer ou préserver AsyncCleanupTask et AsyncStatisticsTask.

Entrée (Control)	**Type**	**Description**
deletePending	Boolean	true = tout, false = préserver stats/cleanup
Entrée (Flow)	**Type**	**Description**
deleteAllCollection	List<List<Boolean>>	[[true]] ou [[false]]
Sortie	**Type**	**Description**
recordsDeleted	Integer	Nombre supprimé
message	String	Message de résultat



AsyncSchedulerSetup

Gestion de la planification du Scheduler depuis le Flow. Actions : GET_INFO, SCHEDULE_EVERY_MINUTE, SCHEDULE_CUSTOM, UPDATE_FREQUENCY_METADATA.

Entrée	Type	Description
action	String	GET_INFO, SCHEDULE_EVERY_MINUTE, SCHEDULE_CUSTOM, UPDATE_FREQUENCY_METADATA
frequencyMinutes	Integer	Pour SCHEDULE_CUSTOM, UPDATE_FREQUENCY_METADATA
jobName	String	Nom du job
useSingleScheduler	Boolean	true = AsyncSchedulerSelfRescheduling

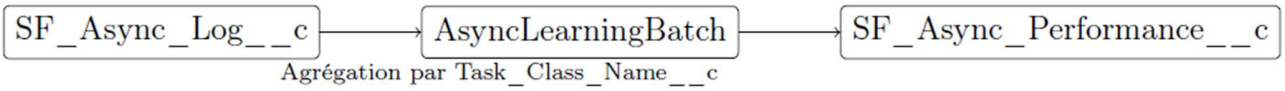
II.6. Tâches intégrées (IAsyncTask)

AsyncCleanupTask

Supprime les SF_Async__c et SF_Async_Log__c exécutés ou annulés depuis plus de Days_To_Keep__c jours. Configurable via Cleanup_Batch_Size__c et Cleanup_CPU_Reserve_Ms__c.

AsyncStatisticsTask

Lance le batch **AsyncLearningBatch** qui agrège les logs SF_Async_Log__c et met à jour SF_Async_Performance__c (Average_CPU__c, Sample_Size__c).



II.7. Runtime et Fallback

Async_Runtime_Config__c (Custom Settings)

Champ	Description
Max_Batch_Size_Override__c	Override temporaire du lot max
Active_Worker_Count__c	Nombre de workers actifs (utilisé par REDUCE_BATCH_SIZE)

AsyncReduceBatchSizeJob

Queueable fallback lorsque EventBus.publish échoue (ex. limite DML). Met à jour le Custom Settings et peut republier l'événement.

III. Manuel Administrateur

Cette section décrit comment un administrateur utilise l'application SMATCH.

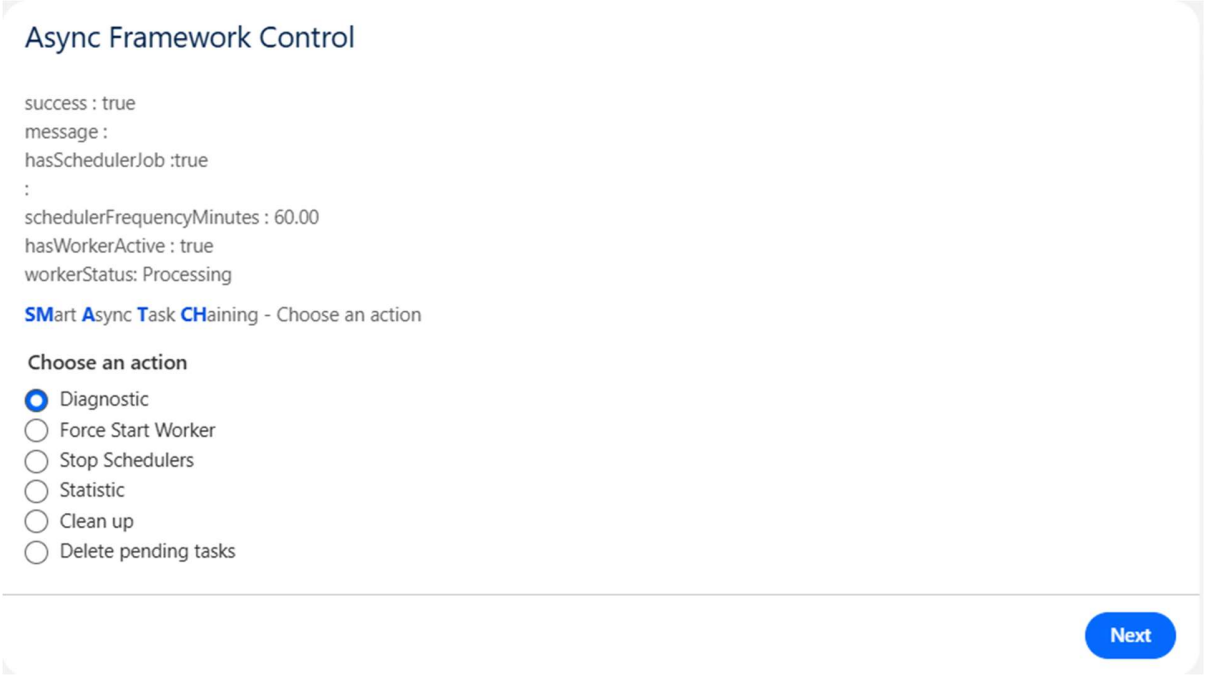
III.1. Accès à l'application Smatch

18. **App Launcher** → rechercher **Smatch**
19. L'application Smatch affiche :

- Le **Flow Async_Framework_Control** (menu principal)
- Optionnellement un **Dashboard** des tâches

III.2. Menu principal du Flow

Au démarrage du Flow, un menu propose les actions suivantes :



Choix	Description
Diagnostic	Affiche l'état complet du framework
Force Start Worker	Relance le Worker et le Watchdog
Stop Schedulers	Arrête tous les schedulers Async
Statistic	Sous-menu Statistiques
Clean up	Sous-menu Nettoyage
Delete Pending Tasks	Suppression des tâches en attente

Capture d'écran suggérée : écran du menu principal (flowDashboard.png si disponible)

III.3. Diagnostic

Async Framework Control

```
=== ASYNC FRAMEWORK DIAGNOSTIC ===
Date/time: 13/02/2026 12:02

PENDING TASKS: 0

RECENT TASKS:
- Daily statistics task | AsyncStatisticsTask | Executed: false | Cancelled: false
- Daily statistics task | AsyncStatisticsTask | Executed: true | Cancelled: false
- Daily Cleanup | AsyncCleanupTask | Executed: false | Cancelled: false
- Daily Cleanup | AsyncCleanupTask | Executed: true | Cancelled: false
- Daily statistics task | AsyncStatisticsTask | Executed: false | Cancelled: true

ERRORS (2h): 0

WORKERS (2h): 5
- Completed (13/02/2026 12:02)
- Completed (13/02/2026 12:01)
- Completed (13/02/2026 12:00)
- Completed (13/02/2026 11:59)
- Completed (13/02/2026 11:58)

SCHEDULERS: 1
- AsyncWorker 20260213120300 | State: WAITING | Next: 13/02/2026 12:03

STATISTICS SCHEDULED: Yes
Frequency: Daily | Next calculation: 13/02/2026 20:06

CLEANUP SCHEDULED: Yes
Frequency: Daily | Next execution: 13/02/2026 19:41

WATCHDOG SCHEDULED: No

WORKER SCHEDULED: Yes
Name: AsyncWorker 20260213120300 | Frequency: 1 min | Next: 13/02/2026 12:03

=== END OF DIAGNOSTIC ===
```

20. Choisir **Diagnostic**
21. Le Flow affiche un rapport contenant :

- Nombre de tâches en attente
- Tâches récentes (executed, cancelled)
- Logs d'erreur des 2 dernières heures
- Workers récents
- Schedulers actifs
- Tâches Statistics et Cleanup planifiées
- État du Watchdog et du Worker

Capture suggérée : résultat du diagnostic (flowdiagnostic.png si disponible)

III.4. Force Start Worker

Utiliser cette action pour :

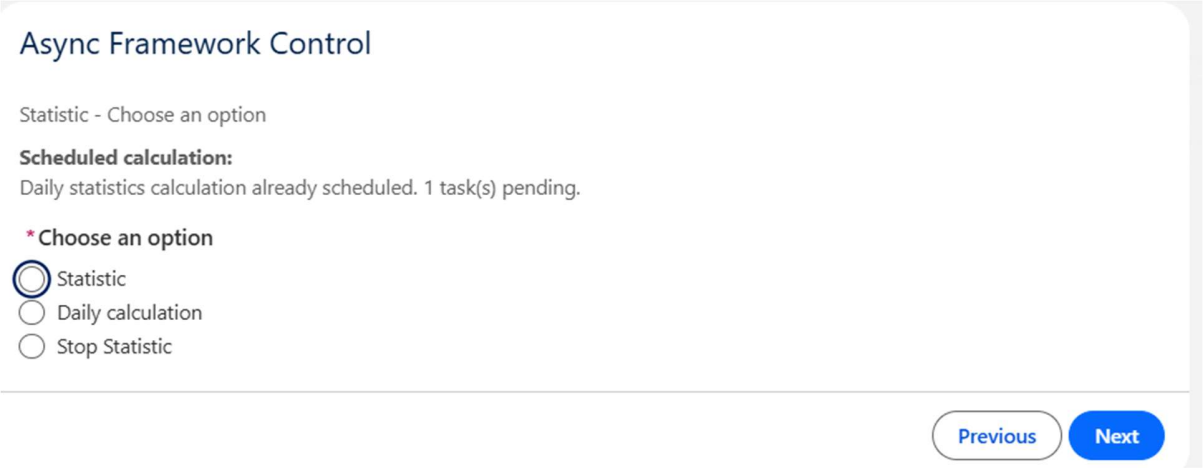
- Relancer le système après un arrêt
- Traiter immédiatement des tâches en attente

Le Flow enqueue le Worker et le Watchdog, et planifie le Scheduler s'il est absent.

III.5. Stop Schedulers

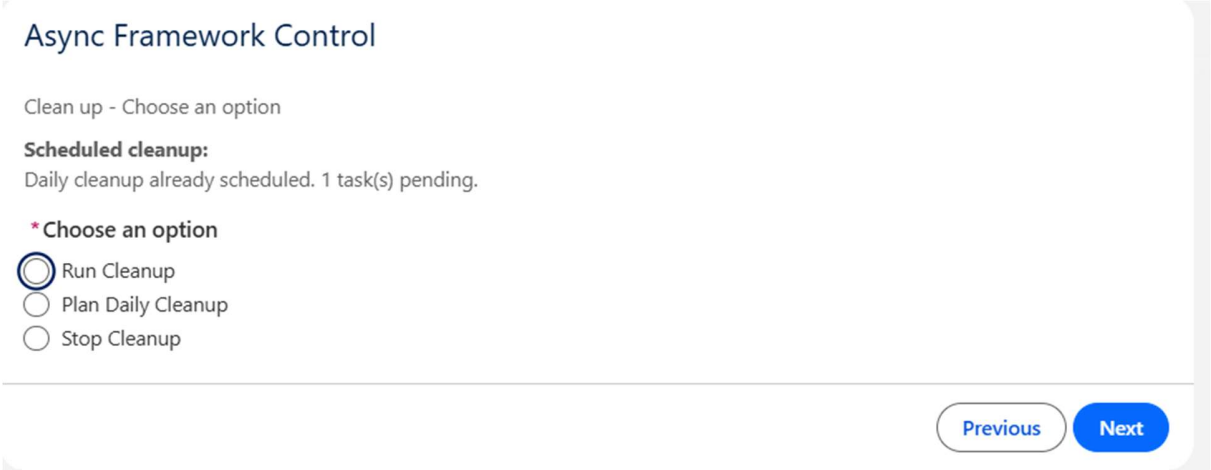
Arrête tous les jobs cron du framework (Async Framework Scheduler, AsyncWorker). Utile pour la maintenance ou la résolution de problèmes.

III.6. Sous-menu Statistic



Action	Description
Start Statistic	Lance immédiatement un calcul de statistiques
Plan Daily Calculation	Planifie un calcul quotidien
Stop Statistic	Annule les tâches de statistiques en cours

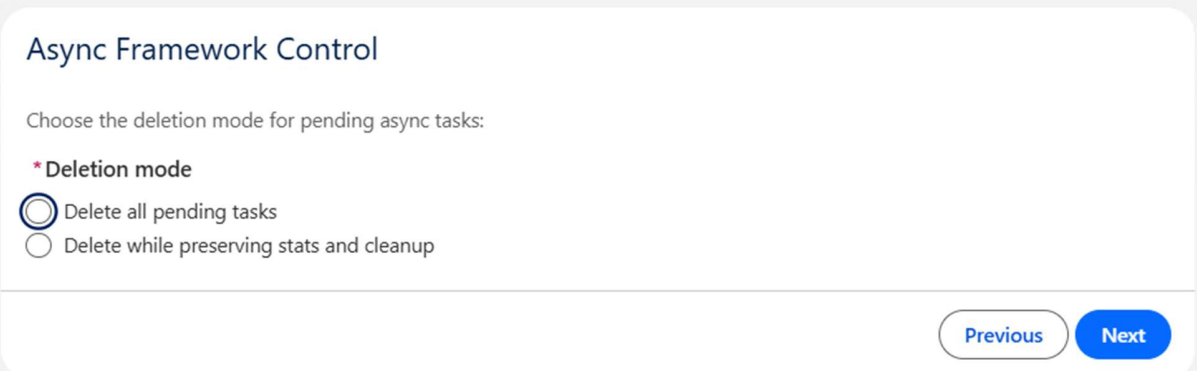
III.7. Sous-menu Clean up



Action	Description
Run Cleanup	Nettoyage immédiat des anciennes données
Plan Daily Cleanup	Planifie un nettoyage quotidien
Stop Cleanup	Annule les tâches de nettoyage planifiées

III.8. Delete Pending Tasks

22. Choisir **Delete Pending Tasks**



23. Sélectionner le mode :

- **Delete All** : supprime toutes les tâches en attente
- **Preserve Stats** : préserve AsyncCleanupTask et AsyncStatisticsTask

24. Confirmer la suppression

III.10. Configuration avancée (Setup)

- **Custom Metadata** : Setup → Custom Metadata Types → Async Framework Configuration → Manage Records → Default Configuration
- **Custom Settings** : Setup → Custom Settings → Async Runtime Config → Manage

- **Remote Site Settings** : pour les tâches avec callout (ex. Calculator)

IV. Manuel du Développeur

Ce chapitre explique comment utiliser le framework SMATCH pour créer des tâches asynchrones. Les exemples **CalculatorCalloutTask** et un exemple conceptuel **CreateAccountTask** illustrent les bonnes pratiques.

IV.1. Interface IAsyncTask

Toute tâche exécutée par SMATCH doit implémenter IAsyncTask :

```
public interface IAsyncTask {
    void run(SF_Async__c contextRecord);
}
```

Entrée : contextRecord – enregistrement SF_Async__c contenant :

- Target_Record_Id__c : ID de l'enregistrement cible
- Additional_Information__c : données JSON ou texte additionnelles
- Task_Class_Name__c : nom de la classe à exécuter

IV.2. Exemple CreateAccountTask (conceptuel)

Tâche simple qui crée un Account. N'existe pas dans le package mais illustre le cas minimal.

```
public class CreateAccountTask implements IAsyncTask {
    public void run(SF_Async__c contextRecord) {
        Account acc = new Account(
            Name = 'Account créé le ' + System.now().format('yyyy-MM-dd HH:mm')
        );
        insert acc;
    }
}
```

Création d'une tâche :

```
SF_Async__c task = new SF_Async__c(
    label__c = 'Créer Account',
    Task_Class_Name__c = 'CreateAccountTask',
    Scheduled_Time__c = System.now(),
    Execution_Order__c = 1,
    Before_Cancelling__c = 3
);
insert task;
EventBus.publish(new Async_WakeUp__e(Event_Type__c = 'WAKEUP'));
```

IV.3. Exemple CalculatorCalloutTask

Prérequis

- Objet Calculator_Result__c (Operand_A__c, Operand_B__c, Result__c)
- Custom Label Calculator_Endpoint (URL de base, ex. https://newton.vercel.app)
- Remote Site Settings pour autoriser le callout

Code

```
public class CalculatorCalloutTask implements IAsyncTask {
    public void run(SF_Async__c contextRecord) {
        Id targetId = contextRecord.Target_Record_Id__c;
        if (targetId == null) {
            throw new AsyncException('Target_Record_Id__c est requis.');
        }
        Calculator_Result__c rec = [SELECT Id, Operand_A__c, Operand_B__c
            FROM Calculator_Result__c WHERE Id = :targetId LIMIT 1];
        Integer a = rec.Operand_A__c != null ? Integer.valueOf(rec.Operand_A__c)
: 0;
        Integer b = rec.Operand_B__c != null ? Integer.valueOf(rec.Operand_B__c)
: 0;
        String baseUrl = System.Label.Calculator_Endpoint;
        if (String.isBlank(baseUrl)) baseUrl = 'https://newton.vercel.app';
        String endpoint = baseUrl + '/api/v2/simplify/' +
EncodingUtil.urlEncode(a+'+'+b, 'UTF-8');
        HttpRequest req = new HttpRequest();
        req.setEndpoint(endpoint);
        req.setMethod('GET');
        req.setTimeout(120000);
        HttpResponse res = new Http().send(req);
        if (res.getStatusCode() != 200) {
            throw new AsyncException('API error: ' + res.getStatusCode());
        }
        Map<String, Object> parsed = (Map<String, Object>)
JSON.deserializeUntyped(res.getBody());
        rec.Result__c = Decimal.valueOf(String.valueOf(parsed.get('result')));
        update rec;
    }
}
```

Flux d'exécution

25. Créer un enregistrement Calculator_Result__c (Operand A, Operand B)
26. Créer une tâche SF_Async__c avec Task_Class_Name__c = 'CalculatorCalloutTask' et Target_Record_Id__c = ID du Calculator Result
27. Publier Async_WakeUp__e(Event_Type__c = 'WAKEUP') ou attendre le Scheduler
28. Le Worker exécute CalculatorCalloutTask.run() qui appelle l'API et met à jour Result__c

Configuration détaillée : voir `docs/CALCULATOR_SETUP_FR.md`

IV.4. Champs SF_Async__c utilisés par le développeur

Champ	Obligatoire	Description
Task_Class_Name__c	Oui	Nom de la classe Apex (ex. CalculatorCalloutTask)
Target_Record_Id__c	Selon la tâche	ID de l'enregistrement cible
Scheduled_Time__c	Oui	Date/heure d'exécution (passé = immédiat)
Execution_Order__c	Non	Ordre relatif (défaut 99)
Before_Cancelling__c	Non	Tentatives max avant annulation (défaut 3)
Tasks_Per_Worker_Limit__c	Non	Limite de tâches par Worker pour cette tâche
label__c	Non	Libellé pour l'affichage
Additional_Information__c	Non	Données additionnelles (JSON, etc.)

IV.5. Mauvaises pratiques à éviter

Pratique	Problème	Solution
DML puis Callout dans la même transaction	Erreur "uncommitted work"	Faire le callout avant tout DML, ou dans une transaction séparée
Tâche trop longue	Timeout CPU	Diviser en plusieurs tâches, réduire la taille du lot
Classe inexistante dans Task_Class_Name__c	TypeException au runtime	Vérifier le nom de classe (avec namespace si packagé)
Pas de gestion d'erreur	Échec silencieux	Lever des exceptions explicites, consulter SF_Async_Log__c
Target_Record_Id__c null pour une tâche qui en a besoin	NullPointerException	Valider les entrées au début de run()
Modifier la configuration sans clearConfig	Valeurs obsolètes	Appeler AsyncFrameworkConfig.clearConfig() après mise à jour metadata

IV.6. Bonnes pratiques

- Utiliser AsyncFrameworkConfig.getBeforeCancelling() et getExecutionOrder() pour les valeurs par défaut
- Tester avec des tâches planifiées dans le passé pour une exécution immédiate
- Consulter SF_Async_Log__c et SF_Async_Performance__c pour le diagnostic
- Pour les callouts : configurer Remote Site Settings et Custom Labels pour les URLs

---

V. Annexe

V.A. Fichiers du package

Classes Apex (extrait)

Classes	Rôle
AggregateResultIterable	Utility – Iterator pour AggregateResult
AggregateResultIterator	Utility – Itérateur pour AggregateResult
AsyncCleanupTask	Task – Nettoyage des logs et statistiques
AsyncFrameworkConfig	Framework – Configuration (metadata)
AsyncFramework_CleanupControl	Flow – Contrôle du nettoyage
AsyncFramework_DeletePendingControl	Flow – Suppression des tâches en attente
AsyncFramework_DeletePendingFlow	Flow – Suppression en attente (Flow)
AsyncFramework_Diagnostic	Flow – Diagnostic du framework
AsyncFramework_ForceStart	Flow – Démarrage forcé du worker
AsyncFramework_StatisticsControl	Flow – Contrôle des statistiques
AsyncFramework_StopSchedulers	Flow – Arrêt des schedulers
AsyncLearningBatch	Task – Apprentissage des performances
AsyncReduceBatchSizeJob	Batch – Réduction de la taille des lots
AsyncScheduler	Scheduler – Scheduler principal
AsyncSchedulerSelfRescheduling	Scheduler – Auto-replanification
AsyncSchedulerSetup	Flow – Configuration du scheduler
AsyncStatisticsTask	Task – Calcul des statistiques
AsyncTaskScheduleInfo	Utility – Infos de planification
AsyncWatchdogQueueable	Watchdog – Surveillance du worker
AsyncWorkerQueueable	Worker – Exécution des tâches asynchrones
AsyncWorkerScheduler	Scheduler – Planification du worker

Objets et métadonnées

Élément	Description
SF_Async__c	Tâches asynchrones
SF_Async_Log__c	Logs d'exécution
SF_Async_Performance__c	Statistiques par classe
Async_WakeUp__e	Platform Event
Async_Runtime_Config__c	Custom Settings
Async_Framework_Configuration__mdt	Custom Metadata
Calculator_Result__c	Objet exemple

Triggers

Trigger	Contexte
AsyncWakeUpTrigger	after insert Async_WakeUp__e

Flows

Flow	Description
Async_Framework_Control	Interface de contrôle principale

V.B. Couverture des tests

Les classes et le trigger disposent de tests unitaires. Résumé de la couverture (état typique) :

Classe	Couverture	Couvertes	Total
AsyncWorkerQueueable	89%	196	221
AsyncFramework_ForceStart	90%	52	58
AsyncFramework_Diagnostic	90%	80	89
AsyncSchedulerSetup	90%	134	149
AsyncFramework_StatisticsControl	91%	43	47
AsyncScheduler	91%	10	11
AsyncFramework_StopSchedulers	91%	29	32
AsyncWorkerScheduler	93%	26	28
AsyncLearningBatch	94%	32	34
AsyncReduceBatchSizeJob	94%	17	18
AsyncStatisticsTask	95%	18	19
AsyncSchedulerSelfRescheduling	95%	21	22
AsyncFramework_DeletePendingControl	95%	56	59
AsyncFramework_CleanupControl	96%	44	46
AsyncCleanupTask	97%	31	32
AsyncFrameworkConfig	98%	93	95
AsyncWatchdogQueueable	100%	28	28
DateTimeFormatUtil	100%	5	5
AggregateResultIterator	100%	11	11
AggregateResultIterable	100%	2	2
AsyncWakeUpTrigger	100%	46	46
AsyncFramework_DeletePendingFlow	100%	8	8
Total	**93%**	**982**	**1060**